

Rapport Technique

Phase 1 : Enrichissement et Normalisation

Feature Engineering pour Transfer Learning

Projet NYC Traffic Prediction

Prédiction de Trafic par Graph Neural Network

Date : 5 janvier 2026

Version : 1.0.0

Script : enrich_graph.py

Table des matières

1	Introduction	3
1.1	Objectif	3
1.2	Contraintes Techniques	3
1.3	Fichiers Source	3
2	Méthodologie de Normalisation	3
2.1	Problème des Features Absolues	3
2.2	Techniques de Normalisation Utilisées	4
2.2.1	Z-Score (Standardisation)	4
2.2.2	Rang Percentile	4
2.2.3	Encodage Cyclique	4
3	Features Générées	4
3.1	Features des Nœuds (Intersections)	4
3.1.1	Calcul du Centre Ville	4
3.1.2	Distance Haversine	5
3.2	Features des Liens (Segments Routiers)	5
3.2.1	Classification Hiérarchique Universelle	5
3.2.2	Capacité Estimée	5
3.3	Features Temporelles	6
4	Architecture du Script	6
4.1	Nettoyage des Données	6
4.1.1	Colonnes Booléennes	6
4.1.2	Gestion des Valeurs Manquantes	6
4.1.3	Sérialisation JSON des Types NumPy	6
4.2	Structure Modulaire	7
4.3	Gestion des Gros Fichiers	7
4.4	Optimisation Mémoire	7
5	Utilisation	8
5.1	Installation des Dépendances	8
5.2	Exécution	8
5.3	Arguments	8
5.4	Fichiers de Sortie	8
5.5	Fichier de Statistiques	8
6	Validation et Tests	9
6.1	Vérifications Automatiques	9
6.2	Métriques de Qualité	9
7	Prochaines Étapes	9
7.1	Phase 2 : Construction du Dataset Edge-Centric	9
7.2	Phase 3 : Architecture du Modèle	9
7.3	Phase 4 : Validation Inductive	9
8	Conclusion	9
A	Code Source Complet	10

B	Formules Mathématiques Détaillées	10
B.1	Distance Haversine	10
B.2	Encodage Cyclique	10

1 Introduction

Ce rapport documente l'implémentation de la **Phase 1** du projet de prédiction de trafic urbain : l'enrichissement et la normalisation des données du graphe routier de New York.

1.1 Objectif

L'objectif principal est de générer des **features relatives et normalisées** qui permettent au modèle GNN d'apprendre des patterns universels, transférables à d'autres villes (ex : Marseille) sans ré-entraînement complet.

1.2 Contraintes Techniques

- **Volume de données** : ~5 GB de données de trafic
- **Transférabilité** : Features indépendantes de l'échelle géographique
- **Performance** : Traitement par chunks pour la gestion mémoire

1.3 Fichiers Source

TABLE 1 – Structure des données d'entrée

Fichier	Contenu	Colonnes clés
nodes.csv	Intersections	node_id, xcoord, ycoord, num_in_links, num_out_links, is_complete (format : 't'/'f')
links.csv	Segments routiers	link_id, begin_node_id, end_node_id, street_length, osm_class
travel_times_2013.csv	Trafic horaire	begin_node_id, end_node_id, datetime, travel_time

Note : Les fichiers utilisent des conventions PostgreSQL (booléens 't'/'f') et peuvent contenir des valeurs manquantes. Le script gère ces cas automatiquement.

2 Méthodologie de Normalisation

Le principe fondamental est de remplacer toutes les **mesures absolues** par des **mesures relatives** qui conservent leur signification indépendamment de la ville.

2.1 Problème des Features Absolues

TABLE 2 – Comparaison features absolues vs relatives

Feature Absolue	Problème	Feature Relative
Distance au centre (km)	Échelle varie par ville	Percentile de centralité
Nombre de commerces	Densité absolue non comparable	Z-score de densité
Longueur (m)	Distribution différente	Percentile de longueur
Heure (0-23)	Discontinuité 23h → 0h	Encodage sin/cos

2.2 Techniques de Normalisation Utilisées

2.2.1 Z-Score (Standardisation)

Pour les distributions approximativement gaussiennes :

$$z = \frac{x - \mu}{\sigma} \quad (1)$$

où μ est la moyenne et σ l'écart-type calculés sur l'ensemble de la ville.

Application : Degrés des nœuds, longueur des segments.

2.2.2 Rang Percentile

Pour les distributions asymétriques ou les valeurs ordinales :

$$p_i = \frac{\text{rank}(x_i)}{N} \quad (2)$$

où N est le nombre total d'observations.

Application : Distance au centre (centralité), capacité estimée.

2.2.3 Encodage Cyclique

Pour les variables périodiques (temps) :

$$x_{\sin} = \sin\left(\frac{2\pi \cdot t}{T}\right) \quad (3)$$

$$x_{\cos} = \cos\left(\frac{2\pi \cdot t}{T}\right) \quad (4)$$

où T est la période (24 pour les heures, 7 pour les jours, 12 pour les mois).

Avantage : Capture la continuité cyclique (23h proche de 0h).

3 Features Générées

3.1 Features des Nœuds (Intersections)

TABLE 3 – Features enrichies des nœuds

Feature	Type	Description
centrality_percentile	[0, 1]	Position relative au centre (0 = centre, 1 = périphérie)
degree_in_normalized	Z-score	Nombre de liens entrants normalisé
degree_out_normalized	Z-score	Nombre de liens sortants normalisé
degree_total_percentile	[0, 1]	Degré total en rang percentile
has_traffic_signal	Binaire	Présence d'un feu de signalisation

3.1.1 Calcul du Centre Ville

Le centre est estimé par le **centroïde médian** des coordonnées :

$$c_x = \text{median}(\{x_i\}_{i=1}^N) \quad (5)$$

$$c_y = \text{median}(\{y_i\}_{i=1}^N) \quad (6)$$

L'utilisation de la médiane (plutôt que la moyenne) rend l'estimation robuste aux outliers géographiques.

3.1.2 Distance Haversine

La distance au centre est calculée via la formule haversine pour tenir compte de la courbure terrestre :

$$d = 2R \cdot \arcsin \left(\sqrt{\sin^2 \left(\frac{\Delta\phi}{2} \right) + \cos(\phi_1) \cos(\phi_2) \sin^2 \left(\frac{\Delta\lambda}{2} \right)} \right) \quad (7)$$

où $R = 6371$ km (rayon terrestre), ϕ = latitude, λ = longitude.

3.2 Features des Liens (Segments Routiers)

TABLE 4 – Features enrichies des liens

Feature	Type	Description
road_hierarchy	{1..6}	Classification hiérarchique universelle
length_percentile	[0, 1]	Longueur relative en rang
length_zscore	Z-score	Longueur normalisée
begin_angle_sin/cos	[-1, 1]	Orientation encodée
start_centrality	[0, 1]	Centralité du nœud de départ
end_centrality	[0, 1]	Centralité du nœud d'arrivée
avg_centrality	[0, 1]	Centralité moyenne du segment
centrality_gradient	[-1, 1]	Direction centre/périphérie
connects_center	Binaire	Segment proche du centre
estimated_capacity	[0, 1]	Capacité relative estimée

3.2.1 Classification Hiérarchique Universelle

Le mapping OSM vers une hiérarchie universelle permet la comparaison entre pays :

TABLE 5 – Mapping OSM vers hiérarchie universelle

Niveau	Classes OSM	Description
1	motorway, trunk	Autoroutes, voies rapides
2	primary	Routes principales
3	secondary	Routes secondaires
4	tertiary	Routes tertiaires
5	residential, living_street	Voies résidentielles
6	service, unclassified	Voies de service

3.2.2 Capacité Estimée

La capacité relative est déduite de la hiérarchie :

$$\text{capacity}(h) = \begin{cases} 1.0 & h = 1 \\ 0.8 & h = 2 \\ 0.6 & h = 3 \\ 0.4 & h = 4 \\ 0.25 & h = 5 \\ 0.15 & h = 6 \end{cases} \quad (8)$$

3.3 Features Temporelles

TABLE 6 – Features temporelles enrichies

Feature	Type	Description
hour_sin/cos	[-1, 1]	Heure encodée cycliquement ($T = 24$)
day_sin/cos	[-1, 1]	Jour de la semaine ($T = 7$)
month_sin/cos	[-1, 1]	Mois de l'année ($T = 12$)
is_weekend	Binaire	Samedi ou dimanche
is_rush_hour	Binaire	7-9h ou 17-19h en semaine
time_period	{1..4}	Matin/Journée/Soir/Nuit

4 Architecture du Script

4.1 Nettoyage des Données

Les fichiers sources contiennent des formats hétérogènes qui nécessitent un pré-traitement :

4.1.1 Colonnes Booléennes

La colonne `is_complete` utilise le format PostgreSQL (`'t'/'f'`) au lieu de valeurs numériques :

```
1 df['is_complete'] = df['is_complete'].astype(str).str.lower()
2 df['is_complete'] = df['is_complete'].map({
3     't': 1, 'true': 1, '1': 1,
4     'f': 0, 'false': 0, '0': 0,
5 }).fillna(0).astype('int8')
```

Listing 1 – Conversion des booléens textuels

4.1.2 Gestion des Valeurs Manquantes

Les colonnes numériques sont chargées en `float32` pour accepter les NaN, puis converties après remplissage :

TABLE 7 – Stratégie de remplissage des valeurs manquantes

Colonne	Valeur par défaut	Justification
num_in_links	0	Nœud isolé
num_out_links	0	Nœud isolé
osm_traffic_controller	'unknown'	Catégorie neutre
osm_class	'unclassified'	Hierarchie minimale
street_length	médiane	Valeur centrale robuste

4.1.3 Sérialisation JSON des Types NumPy

Les statistiques de normalisation contiennent des types NumPy (`int64`, `float64`) non sérialisables en JSON standard. Un encodeur personnalisé effectue la conversion :

```
1 class NumpyEncoder(json.JSONEncoder):
2     def default(self, obj):
```

```

3         if isinstance(obj, np.integer):
4             return int(obj)
5         elif isinstance(obj, np.floating):
6             return float(obj)
7         elif isinstance(obj, np.ndarray):
8             return obj.tolist()
9         return super().default(obj)
10
11 # Utilisation
12 json.dump(stats, f, cls=NumpyEncoder)

```

Listing 2 – Encodeur JSON pour NumPy

4.2 Structure Modulaire

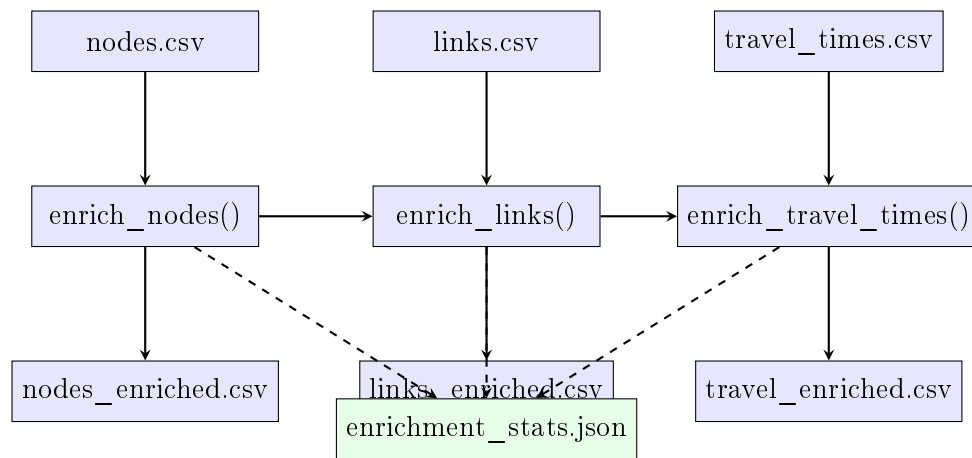


FIGURE 1 – Pipeline de traitement du script enrich_graph.py

4.3 Gestion des Gros Fichiers

Pour le fichier `travel_times_2013.csv` (~5 GB), le traitement se fait par **chunks** :

```

1 CHUNK_SIZE = 500_000 # 500k lignes par chunk
2
3 for chunk in pd.read_csv(input_path, chunksize=CHUNK_SIZE):
4     chunk = process_travel_times_chunk(chunk)
5     chunk.to_csv(output_path, mode='a', header=first_chunk)
6     first_chunk = False

```

Listing 3 – Traitement par chunks

4.4 Optimisation Mémoire

Les types de données sont spécifiés pour minimiser l’empreinte mémoire :

```

1 LINKS_DTYPES = {
2     'link_id': 'int64',
3     'street_length': 'float32', # 4 bytes vs 8
4     'osm_class': 'category', # Hash vs string
5     'begin_angle': 'float32',
6 }

```

Listing 4 – Types optimisés

5 Utilisation

5.1 Installation des Dépendances

```
1 pip install pandas numpy scipy
```

Listing 5 – Requirements

5.2 Exécution

```
1 # Usage basique
2 python enrich_graph.py --data-dir ./travel_times_2013 --output-dir ./
   enriched_data
3
4 # Avec features de voisinage (optionnel, coûteux)
5 python enrich_graph.py -d ./data -o ./output --compute-neighbors
```

Listing 6 – Commande d'exécution

5.3 Arguments

TABLE 8 – Arguments du script

Argument	Défaut	Description
-d, -data-dir	./travel_times_2013	Répertoire des données sources
-o, -output-dir	./enriched_data	Répertoire de sortie
-compute-neighbors	False	Calcul des features de voisinage

5.4 Fichiers de Sortie

1. nodes_enriched.csv : Nœuds avec features de centralité
2. links_enriched.csv : Liens avec features topologiques
3. travel_times_enriched.csv : Données temporelles enrichies
4. enrichment_stats.json : Statistiques de normalisation

5.5 Fichier de Statistiques

Le fichier JSON contient les paramètres nécessaires pour appliquer les mêmes normalisations lors de l'inférence sur une nouvelle ville :

```
1 {
2   "nodes": {
3     "city_center": {"x": -73.98, "y": 40.75},
4     "degree_in": {"mean": 2.3, "std": 1.1},
5     "degree_out": {"mean": 2.3, "std": 1.1}
6   },
7   "links": {
8     "street_length": {"mean": 150.5, "std": 89.2},
9     "road_hierarchy_distribution": {"1": 0.05, "2": 0.12, ...}
10  },
11  "metadata": {
12    "created_at": "2026-01-05T...",
13    "osm_class_hierarchy": {...}
14  }
15 }
```

6 Validation et Tests

6.1 Vérifications Automatiques

Le script génère des logs de validation :

```
1 2026-01-05 17:45:00 - INFO - Centre ville estime: (-73.98, 40.75)
2 2026-01-05 17:45:01 - INFO - Distribution hierarchie: {1: 0.05, 2: 0.12, ...}
3 2026-01-05 17:45:02 - INFO - Taux de correspondance liens: 98.5%
```

Listing 8 – Exemple de sortie

6.2 Métriques de Qualité

- **Taux de correspondance** : Pourcentage de liens du fichier travel_times matchés
- **Distribution de hiérarchie** : Vérification de la couverture des types de routes
- **Distance max au centre** : Validation de l'estimation du centre

7 Prochaines Étapes

7.1 Phase 2 : Construction du Dataset Edge-Centric

Transformation du graphe en **Line Graph** (Dual Graph) :

- Les routes deviennent des nœuds
- Les intersections deviennent des arêtes
- Compatible avec les architectures GNN standard (GraphSAGE, GAT)

7.2 Phase 3 : Architecture du Modèle

1. Input Layer : Encodage des features enrichies
2. Message Passing ($\times 3$) : Agrégation spatiale
3. Readout Layer : MLP pour régression de la vitesse

7.3 Phase 4 : Validation Inductive

- **Train** : Brooklyn, Manhattan
- **Test** : Queens (zone non vue)
- **Objectif** : Si validation Queens réussie $\Rightarrow$ transfert Marseille viable

8 Conclusion

Ce rapport a présenté l'implémentation complète de la Phase 1 d'enrichissement du graphe routier. Les contributions principales sont :

1. **Normalisation universelle** : Toutes les features sont relatives, permettant le transfer learning
2. **Encodage cyclique** : Capture correcte de la temporalité (heures, jours, mois)
3. **Classification hiérarchique** : Mapping OSM indépendant du pays
4. **Scalabilité** : Traitement par chunks pour fichiers de 5GB+

5. Reproductibilité : Statistiques sauvegardées pour inférence

Le script `enrich_graph.py` est prêt à être exécuté sur l'environnement local avec les données complètes.

A Code Source Complet

Le script complet `enrich_graph.py` est fourni séparément. Les fonctions principales sont :

- `enrich_nodes()` : Enrichissement des intersections
- `enrich_links()` : Enrichissement des segments
- `process_travel_times_chunk()` : Traitement temporel par chunk
- `compute_neighborhood_features()` : Features de voisinage (optionnel)

B Formules Mathématiques Détaillées

B.1 Distance Haversine

Implémentation vectorisée NumPy :

```
1 def haversine_distance(lon1, lat1, lon2, lat2):
2     R = 6371 # km
3     dlon = np.radians(lon2 - lon1)
4     dlat = np.radians(lat2 - lat1)
5     a = np.sin(dlat/2)**2 + \
6         np.cos(np.radians(lat1)) * np.cos(np.radians(lat2)) * \
7         np.sin(dlon/2)**2
8     return 2 * R * np.arcsin(np.sqrt(a))
```

B.2 Encodage Cyclique

```
1 def encode_cyclic(values, period):
2     angle = 2 * np.pi * values / period
3     return np.sin(angle), np.cos(angle)
```